

Research and Integration Guide: aiortc and PyNvVideoCodec

1 Overview

The **TorchWindow 2.0** streaming architecture proposes using Python's `aiortc` library to handle WebRTC sessions and NVIDIA's **PyNvVideoCodec** to perform GPU-accelerated video encoding. This report explains how each library works based on official documentation and source code and validates how they can be integrated for our requirements (lossless GPU-based encoding and streaming). A step-by-step integration guide is provided.

Our use case

- Headless simulation produces CUDA tensors representing frames (RGB or RGBA). These frames must be encoded losslessly on the GPU and streamed in real time to browser clients using WebRTC.
- Clients can request different viewports (zoom/pan) and resolutions. Encoding must remain lossless.

2 Understanding `aiortc`

`aiortc` is a pure-Python implementation of WebRTC. The key components for video streaming are:

1. `MediaStreamTrack` – abstract base class representing a media source. Its `recv()` coroutine must return either an `AudioFrame`, a `VideoFrame` or a `Packet`; the documentation explicitly states this union ¹. Returning a `Packet` allows you to supply pre-encoded data.
2. `RTCPeerConnection` and `RTCRTpsender` – When you call `addTrack()` on a peer connection, it returns an `RTCRTpsender`. The sender is responsible for encoding and packetizing frames from the track and sending them over RTP.
3. **Internal encoder/packetizer** – `RTCRTpsender` obtains frames from the track via `await track.recv()`. The `_next_encoded_frame()` method uses a codec-specific `Encoder` (e.g., `H264Encoder`) to convert a raw `av.Frame` to payloads. If the incoming data is already pre-encoded (i.e., an `av.Packet`), the sender calls `encoder.pack(data)` instead of encoding ². This is the mechanism that enables external encoders.
4. **Packetization** – The built-in `H264Encoder.pack()` function accepts an `av.Packet`, splits the H.264 bitstream into RTP-sized payloads and computes the RTP timestamp from the packet's PTS and time base ³. For pre-encoded frames to work, we must supply an `av.Packet` with valid `packet.pts` and `packet.time_base`.

5. **Codec support** – As of aiortc 1.13 (June 2024), the library only provides built-in encoders/decoders for VP8 and H.264 ⁴. However, by returning pre-encoded packets, we can transmit other codecs (HEVC, AV1) if the browser understands them and we correctly set the SDP parameters and RTP payload type. Implementing this requires manual negotiation and may not be supported by all browsers.

Practical implications for our use case

- To stream lossless video, we need to encode frames externally (using PyNvVideoCodec) and feed the encoded bitstreams into `aiortc` as `av.Packet` objects.
- The `MediaStreamTrack.recv()` coroutine will become our producer of pre-encoded frames.
- We must fill in the packet's `pts` and `time_base` so `aiortc` can compute correct RTP timestamps for each frame.
- Because `aiortc`'s built-in packetizer targets H.264, sending HEVC/AV1 frames may require writing a custom `Encoder.pack()` to split bitstreams according to RFC 7798 (HEVC) or draft specifications for AV1. This guide therefore focuses on the supported H.264 path.

3 Understanding PyNvVideoCodec

PyNvVideoCodec is a Python binding over NVIDIA's Video Codec SDK. The API makes it possible to create encoders/decoders that run on the GPU and accept input from device memory.

Creating an encoder

Encoders are created with the `CreateEncoder()` function. According to the API guide, you must provide width, height, the surface format, a flag indicating whether the input is host memory or device memory, and optional parameters ⁵. A typical call looks like this:

```
import PyNvVideoCodec as nvc

encoder = nvc.CreateEncoder(
    width=1920,
    height=1080,
    format="NV12",          # or "YUV444", "ARGB", etc.
    usecpuinputbuffer=False, # False → input from device memory (CUDA)
    **config_params        # optional parameters
)
```

Important parameters include:

Parameter	Purpose
<code>gpuid</code>	GPU ordinal (defaults to 0).

Parameter	Purpose
<code>width</code> , <code>height</code>	Resolution of the frames.
<code>format</code>	Surface format (<code>NV12</code> , <code>YUV444</code> , <code>ARGB</code> , etc.) ⁶ . For lossless 4:4:4 encoding, <code>YUV444</code> or <code>ARGB</code> should be used.
<code>usecpuinputbuffer</code>	<code>False</code> means the input is device memory; <code>True</code> for host memory ⁷ .
<code>**kwargs</code>	Optional parameters like <code>codec</code> , <code>bitrate</code> , <code>fps</code> , <code>constqp</code> , <code>tuning_info</code> , etc.

Encoding frames

The typical workflow is:

1. **Create the encoder.**
2. **Feed raw frames** via `Encode()` . The input can be:
 3. A 1-D byte array containing NV12/YUV data ⁸ .
 4. An object implementing the **CUDA Array Interface** for each plane (allows encoding directly from CUDA device memory) ⁹ .
 5. A NCHW tensor (batch=1, channel=1) representing NV12 data ¹⁰ .
6. **Retrieve encoded bitstream** – `Encode()` returns an array of bytes containing the encoded NAL units ¹¹ .
7. **Flush the encoder** using `EndEncode()` to retrieve any remaining data ¹² .

GPU memory input

To avoid host-device copies, we feed CUDA tensors directly. The API guide shows how to implement a class with a `cuda()` method returning an array for each plane ¹³ . Alternatively, for NV12 you can pass a 2-D `torch.cuda.ByteTensor` of shape `[1620, 1920]` (height × width × 1.5 due to NV12's luma+chroma packing) and the encoder will interpret it correctly ¹⁴ .

Configuring lossless encoding

PyNvVideoCodec exposes optional parameters to achieve lossless encoding. These include:

- `codec` : choose `h264` , `hevc` or `av1` depending on GPU support.
- `constqp` : set a constant QP value; lossless mode sets QP 0.
- `tuning_info="lossless"` : enables lossless tuning and disables rate control.
- `preset` : choose from `P1` – `P7` (`P7` being slowest/highest quality) ¹⁵ .
- `pixel_format` / `format` : use `YUV444` or `ARGB` to maintain full chroma resolution.
- `rateControlMode` : set to `constqp` for constant QP or disable rate control.

Because browsers currently handle H.264 more reliably than AV1/HEVC in WebRTC, this guide uses **H.264** with `constqp=0` , `tuning_info='lossless'` , and `format='YUV444'` to achieve visually lossless encoding.

Limitations

- Lossless H.264 4:4:4 decode support varies across browsers. Some browsers may downgrade the stream to 4:2:0 or fail to play. You must test browser compatibility.
- PyNvVideoCodec's AV1/HEVC support is new; not all GPUs support these codecs. The B200/Blackwell platform should support AV1 and HEVC.

4 Integration Strategy

Our goal is to encode GPU frames using PyNvVideoCodec and feed the resulting bitstreams into `aiortc` for WebRTC streaming. The high-level steps are:

1. **Convert and prepare the simulation frame.** Starting with a CUDA tensor representing the simulation image ($H \times W \times 3$ RGB floats or bytes):
2. Convert from RGB(A) to **YUV444** or **ARGB** on the GPU. A small CUDA kernel or a Torch/CuPy operation can perform this conversion efficiently. The result is a contiguous buffer meeting PyNvVideoCodec's format.
3. Ensure the buffer shape matches the expected luma+chroma layout. For NV12, the height must be $1.5 \times H$; for YUV444, it's $3 \times H$.
4. **Encode on the GPU** using PyNvVideoCodec:

```
import PyNvVideoCodec as nvc
# Suppose frame_yuv is a torch.cuda.ByteTensor with shape [H*3, W] for YUV444
encoder = nvc.CreateEncoder(
    width=W,
    height=H,
    format="YUV444",          # full chroma for lossless
    usecpuinputbuffer=False,  # input is device memory
    codec="h264",
    constqp=0,
    tuning_info="lossless",
    preset="p7"               # highest quality (slowest)
)

bitstream = encoder.Encode(frame_yuv)
# bitstream is a bytes-like object containing one or more NAL units
```

1. **Wrap the encoded bytes into an** `av.Packet`. `aiortc` uses [PyAV](#) for frame/packet representations. We need to set the packet's presentation timestamp (`pts`) and time base:

```
import av
from aiortc.codecs.h264 import VIDEO_TIME_BASE
```

```
packet = av.Packet(bitstream)
packet.pts = frame_index # monotonically increasing integer
packet.time_base = VIDEO_TIME_BASE # typically 1/90000 for video 3
```

1. Create a custom `MediaStreamTrack` subclass that yields `Packet` objects:

```
from aiortc import MediaStreamTrack

class EncodedVideoTrack(MediaStreamTrack):
    kind = "video"

    def __init__(self, width, height):
        super().__init__()
        self.width = width
        self.height = height
        self.frame_index = 0
        # set up encoder as above

    async def recv(self):
        # get CUDA tensor for current frame and convert to YUV444
        frame_yuv = get_simulation_frame_cuda(self.frame_index)
        bitstream = encoder.Encode(frame_yuv)
        packet = av.Packet(bitstream)
        packet.pts = self.frame_index
        packet.time_base = VIDEO_TIME_BASE
        self.frame_index += 1
        return packet # returning Packet triggers encoder.pack in RTCRtpSender
```

When the track returns a `Packet`, `RTCRtpSender` bypasses the internal encoding and calls `encoder.pack(packet)` 2 to split the bitstream into RTP payloads and compute the timestamp. This ensures that our GPU-encoded frames are sent without re-encoding.

1. **Configure WebRTC session.** On the server side:

```
pc = RTCPeerConnection()
track = EncodedVideoTrack(width=W, height=H)
pc.addTrack(track)
# create SDP offer/answer as usual and handle ICE negotiation via aiortc
```

1. **Signaling and client code.** Use `aiohttp` to serve an HTML/JS client. The JS should create a `RTCPeerConnection`, fetch the SDP offer/answer from the server, attach the video track, and display it in a `<video>` element.

2. **Handling viewports.** For each client's viewport, maintain a separate `EncodedVideoTrack` instance that crops and resizes the simulation tensor before encoding. This allows different resolutions without re-encoding a single high-resolution stream.

Considerations

- **Timestamp management** – The `pts` value must increase by a constant amount per frame to reflect the frame rate; e.g., if you use `VIDEO_TIME_BASE = 1/90000` (90 kHz), then `pts = frame_index * (90000 / fps)`. `aiortc` uses this to compute RTP timestamps.
- **Keyframes** – Lossless H.264 still uses IDR frames. To force periodic IDR frames, you may expose a parameter to `PyNvVideoCodec` or call `encoder.EndEncode()` periodically.
- **SDP negotiation** – The browser must advertise support for H.264 4:4:4 (profile `High444Predictive`) or at least baseline; many browsers only accept 4:2:0. You may need to adjust to 4:2:0 if compatibility issues arise.
- **Browser compatibility** – Test in Chrome with the [WebCodecs API](#) or ensure the browser can decode H.264 lossless.

5 Validation of the Integration Approach

From the aiortc side:

- `MediaStreamTrack.recv()` may yield a pre-encoded `av.Packet`. The documentation clarifies that `recv()` returns either a `Frame` or a `Packet` ¹. In `RTCRTpsender._next_encoded_frame()`, if the object is not an `av.Frame` then `encoder.pack()` is called to packetize it ². This ensures aiortc will forward our pre-encoded bitstream without transcoding.
- The built-in `H264Encoder.pack()` splits an `av.Packet` into RTP payloads and derives the RTP timestamp by converting `packet.pts` using `VIDEO_TIME_BASE` ³. As long as we set `packet.pts` and `packet.time_base` correctly, the packetizer produces valid RTP packets.

From the PyNvVideoCodec side:

- The API allows encoding from CUDA tensors via the `usecpuinputbuffer=False` flag ⁷ and accepts objects implementing the CUDA Array Interface ⁹. This matches our requirement to avoid GPU→CPU copies.
- The `Encode()` method returns an array of encoded bitstream which we can wrap into `av.Packet` ¹¹. The `EndEncode()` flush ensures no trailing frames remain ¹².
- Optional parameters such as `constqp=0` and `tuning_info='lossless'` enable lossless H.264 encoding. Setting the format to `YUV444` avoids chroma subsampling and maintains full fidelity ⁶.

Limitations & Workarounds:

- **Codec support** – aiortc currently only provides built-in support for VP8/H.264. We focus on H.264. Sending AV1/HEVC would require implementing a custom packetizer or patching aiortc.
- **Browser support** – Many browsers decode only 4:2:0 H.264. If 4:4:4 lossless fails, consider down-converting to 4:2:0 or using hardware decode on the client via WebCodecs.
- **Time bases** – Ensure that `packet.time_base` matches the encoder's output time base. The default `VIDEO_TIME_BASE` is 1/90000; adjust if using a non-standard frame rate.

6 Integration Steps Summary

1. Install dependencies:

```
pip install aiortc aiohttp av torch cupy pynvvideocodec # NVENC support
```

1. **Set up the encoder** with `PyNvVideoCodec` using `lossless` parameters. Use `YUV444` or `ARGB` surface format and `usecpuinputbuffer=False` ¹⁶.
2. **Implement GPU-side color conversion** from simulation tensor to YUV444 or NV12.
3. **Create a `MediaStreamTrack` subclass** that encodes each frame with `PyNvVideoCodec` and returns an `av.Packet` with proper `pts` / `time_base`.
4. **Add the track to `aiortc.RTCPeerConnection`**, negotiate SDP using `RTCPeerConnection.createOffer()` / `setLocalDescription()` and deliver it to the client via an `aiohttp` server.
5. **Implement an HTML client** that uses WebRTC (JavaScript) to receive the video stream and display it in a `<video>` element. The UI should also show stream statistics, window name, FPS, and latency by polling `/metrics.json`.
6. **Use data channels** to send control messages (pause, resume, zoom/pan) back to the server. The server modifies viewport parameters accordingly and encodes the requested view.
7. **Test across browsers.** Ensure the chosen H.264 profile and color format are supported. Fallback to a supported profile if necessary.

7 Conclusion

The combination of **aiortc** and **PyNvVideoCodec** can support a GPU-accelerated, lossless video streaming pipeline. `aiortc` can ingest pre-encoded frames because `MediaStreamTrack.recv()` is allowed to return `av.Packet` objects ¹, and `RTCRtpSender` will packetize them using the `pack` method ².

PyNvVideoCodec provides an easy way to create a hardware encoder and feed it CUDA tensors; the `Encode` method returns encoded bitstreams and supports device memory input ⁸ ⁹. By converting simulation frames to YUV444 on the GPU, configuring PyNvVideoCodec for lossless H.264, wrapping the output into `av.Packet` objects with correct timestamps, and yielding them in a custom `MediaStreamTrack`, we can stream lossless video over WebRTC.

Because aiortc's packetizer is designed for H.264, sticking with H.264 for now ensures compatibility. Extending the approach to AV1 or HEVC would require custom packetization and SDP negotiation. Nonetheless, the method described here meets our requirements for headless, lossless, GPU-accelerated streaming and provides a clear path for implementation.

¹ API Reference — aiortc documentation

<https://aiortc.readthedocs.io/en/latest/api.html>

² raw.githubusercontent.com

<https://raw.githubusercontent.com/aiortc/aiortc/main/src/aiortc/rtrtpsender.py>

³ raw.githubusercontent.com

<https://raw.githubusercontent.com/aiortc/aiortc/main/src/aiortc/codecs/h264.py>

⁴ raw.githubusercontent.com

https://raw.githubusercontent.com/aiortc/aiortc/main/src/aiortc/codecs/__init__.py

⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ PyNvVideoCodec API Programming Guide - NVIDIA Docs

<https://docs.nvidia.com/video-technologies/pynvvideocodec/pynvc-api-prog-guide/index.html>